[illegible]

[illegible] to help us do [illegible] to learn [illegible]

[illegible]

[illegible] multiples [illegible] should be printed on [illegible]

Input Format

A single integer n

Constraints

$2 \leq n \leq 20$

Output Format

Print **10** lines of output; each line i (where $1 \leq i \leq 10$) contains the **result** of $n \times i$ in the form:

n x i = result

Sample Input

2

Sample Output

2 x 1 = 2

2 x 2 = 4

2 x 3 = 6

2 x 4 = 8

2 x 5 = 10

2 x 6 = 12

2 x 7 = 14

2 x 8 = 16

2 x 9 = 18

2 x 10 = 20

Answer: (penalty regime: 0 %)

```
#include<stdio.h>
int main()
{
    int n;
    scanf("%d",&n);
    for(int i = 1; i <=10; i++)
    {
        printf("%d x %d = %d\n", n, i, n
    }
}
```

	Input	Expected	Got	
✓	2	2 x 1 = 2 2 x 2 = 4 2 x 3 = 6 2 x 4 = 8 2 x 5 = 10 2 x 6 = 12 2 x 7 = 14 2 x 8 = 16 2 x 9 = 18 2 x 10 = 20	2 x 1 = 2 2 x 2 = 4 2 x 3 = 6 2 x 4 = 8 2 x 5 = 10 2 x 6 = 12 2 x 7 = 14 2 x 8 = 16 2 x 9 = 18 2 x 10 = 20	✓

Passed all tests! ✓

[illegible]

Sample Input 1

1

Sample Output 1

2

Explanation 1

1. Cannot use item *1* because $k = 1$ and $sum = k$ has to be avoided at any time.
2. Hence, max total is achieved by $sum = 0 + 2 = 2$.

Sample Case 2

Sample Input For Custom Testing

Sample Input 2

3
3

Sample Output 2

5

Explanation 2

2 + 3 = 5, is the best case for maximum nutrients.

Answer: (penalty regime: 0 %)

```
#include<stdio.h>
int main()
{
    int n,k,t=0;
    scanf("%d%d",&n,&k);
    for(int i=1;i<=n;i++){
        t=(t+i)%1000000007;
        if(t==k){
            t -=1;
        }
    }
    printf("%d",t);
}
```

	Input	Expected	Got	
✓	2 2	3	3	✓
✓	2 1	2	2	✓
✓	3 3	5	5	✓

Passed all tests! ✓

Sample Output 0

5

Explanation 0

Factoring $n = 10$ we get $\{1, 2, 5, 10\}$. We then return the $p = 3^{rd}$ factor as our answer.

Sample Input 1

10

5

Sample Output 1

0

Explanation 1

Factoring $n = 10$ we get $\{1, 2, 5, 10\}$. There are only 4 factors and $p = 5$. We return 0 as our answer.

Sample Input 2

1

1

Sample Output 2

1

Explanation 2

Factoring $n = 1$ we get $\{1\}$. We then return the $p = 1^{st}$ factor as our answer.

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>
int main()
{
        int n,p,c=0;
        scanf("%d%d",&n,&p);
        for(int i=1;i<=n;i++){
                if(n%i==0){
                        c++;
                }
                if(c==p){
                        printf("%d",i);
                        break;
                }
        }
        if(c!=p){
                printf("%d",0);
        }
        return 0;
}
```

	Input	Expected	Got	
✓	10 3	5	5	✓
✓	10 5	0	0	✓
✓	1 1	1	1	✓

Passed all tests! ✓